

@ViewChild e @ViewChildren são decoradores do Angular usados para obter referências de elementos, componentes ou diretivas dentro do template (HTML) de um componente para manipulá-los na classe (TypeScript). Eles funcionam de forma semelhante ao document.getElementById no JavaScript, permitindo acessar propriedades e métodos de filhos. [1, 2]

Principais Diferenças e Uso:

- **@ViewChild:** Retorna apenas a **primeira** ocorrência de um elemento que corresponda ao seletor. [3]
- **@ViewChildren:** Retorna uma lista (QueryList) contendo **todas** as ocorrências que correspondem ao seletor, sendo atualizada automaticamente se o DOM mudar. [3, 4, 5]
- **Contexto:** São usados para elementos definidos dentro da própria visão do componente (template). [6]
- **Signals (Angular 19+):** O viewChild agora pode ser usado como um sinal reativo, dispensando o gerenciamento manual de mudanças, sendo uma alternativa estável e mais eficiente aos decoradores tradicionais. [5, 7]

Exemplo Prático de Uso:

```
import { Component, ViewChild, ViewChildren, ElementRef, QueryList, AfterViewInit } from '@angular/core';
```

```
@Component({
  selector: 'app-exemplo',
  template: `
    <input #meuInput type="text">
    <div class="item">Item 1</div>
    <div class="item">Item 2</div>
  `
})

export class ExemploComponent implements AfterViewInit {
  // @ViewChild para um único elemento
  @ViewChild('meuInput') inputRef!: ElementRef;

  // @ViewChildren para múltiplos elementos
  @ViewChildren('item') itemsRef!: QueryList<ElementRef>;

  ngAfterViewInit() {
```

```
// Acessando o elemento após a inicialização da view
this.inputRef.nativeElement.focus();

console.log(this.itemsRef.length); // Exibe 2
}
}
```

Pontos Importantes:

- **{ static: true/false }**: Pode ser passado como segundo argumento. true permite acessar a referência no ngOnInit, mas o elemento não pode estar dentro de um *ngIf ou *ngFor.
- **ContentChild/ContentChildren**: Diferente do ViewChild, estes servem para acessar elementos projetados via <ng-content>.
- **Ciclo de Vida**: As referências geralmente só estão disponíveis a partir do ngAfterViewInit. [3, 6, 8, 9]

Enquanto o ViewChild olha para o que está **dentro** do seu próprio template, o @ContentChild e o @ContentChildren olham para o que é **projetado** dentro do seu componente através de <ng-content>.

Imagine que você criou um componente de "Card" e o usuário coloca um botão dentro dele no HTML principal. Para o "Card" acessar esse botão, ele precisa usar esses decoradores.

Diferenças Principais:

- **@ContentChild**: Pega a primeira instância do elemento projetado.
- **@ContentChildren**: Pega todos os elementos projetados como uma QueryList.
- **Ciclo de Vida**: As referências só ficam disponíveis no hook **ngAfterContentInit**.

Exemplo Prático

Vamos criar um componente PaiComponent que projeta conteúdo para dentro de um CardComponent.

1. O Componente que recebe o conteúdo (Card)

É aqui que usamos os decoradores para "capturar" o que vem de fora.

```
import { Component, ContentChild, ContentChildren, QueryList, ElementRef, AfterContentInit } from
 '@angular/core';
```

```
@Component({
  selector: 'app-card',
  template: `
```

```

<div style="border: 1px solid black; padding: 10px;">
  <ng-content></ng-content> <!-- Onde o conteúdo cairá -->
</div>
,
})

export class CardComponent implements AfterContentInit {
  // Captura um único elemento com a tag #titulo
  @ContentChild('titulo') tituloRef!: ElementRef;

  // Captura todos os parágrafos projetados
  @ContentChildren('paragrafo') paragrafos!: QueryList<ElementRef>;

  ngAfterContentInit() {
    // Agora podemos manipular os elementos projetados
    this.tituloRef.nativeElement.style.color = 'blue';

    console.log(`Total de parágrafos projetados: ${this.paragrafos.length}`);
  }
}

```

2. Usando o componente (Onde a projeção acontece)

Aqui definimos o que será enviado para dentro do card.

```

<app-card>
  <h1 #titulo>Título do Card</h1> <!-- Capturado por @ContentChild -->

  <p #paragrafo>Primeiro parágrafo.</p> <!-- Capturados por @ContentChildren -->
  <p #paragrafo>Segundo parágrafo.</p>
</app-card>

```

Quando usar um ou outro?

Decorador	Onde ele procura?	Exemplo de uso
-----------	-------------------	----------------

ViewChild	No HTML do próprio componente.	Pegar um input que você escreveu no template.
ContentChild	No conteúdo que alguém colocou entre as tags do seu componente.	Criar abas (Tabs) onde cada aba é projetada pelo usuário do componente.

Você gostaria de ver como implementar isso usando a nova API de **Signals** (contentChild) que o Angular introduziu recentemente?